Estilos arquitectónicos y Arq. de Software

Arquitectura de Software

Especificación de Requisitos del Sistema (SRS)



En este camino hay mucho por hacer.

¿Comenzamos a programar para terminar lo antes posible?

¿Cuáles serían los riesgos?

Sistema instalado y funcionando

Arquitectura de Software

Especificación de Requisitos del Sistema (SRS)

Estas fases no se aplican en Cascada.



- Arquitectura de Software
- Diseño detallado
- Implementación
- Verificación

Sistema instalado y funcionando

Arquitectura de Software

- Los sistemas complejos están compuestos de subsistemas que interactúan bajo el control de un diseño del sistema.

- Arquitectura de Software
  - Los subsistemas que componen el sistema,
  - las interfaces y
  - las reglas de interacción entre ellos.

Definición de Arquitectura de Software

- Una arquitectura de software para un sistema es la estructura o estructuras del sistema, que consta de elementos, sus propiedades visibles externamente y las relaciones entre ellos.

A software architecture for a system is the structure or structures of the system, which consist of elements, their externally visible properties, and the relationships among them.

Documenting software architectures, views and beyond

Valor de la Arquitectura

- Ventajas de diseñar y documentar explícitamente una arquitectura de software:
  - Comunicación entre interesados
  - Decisiones tempranas de diseño
  - Reúso a gran escala

Bass, et al, Software Architecture in Practice

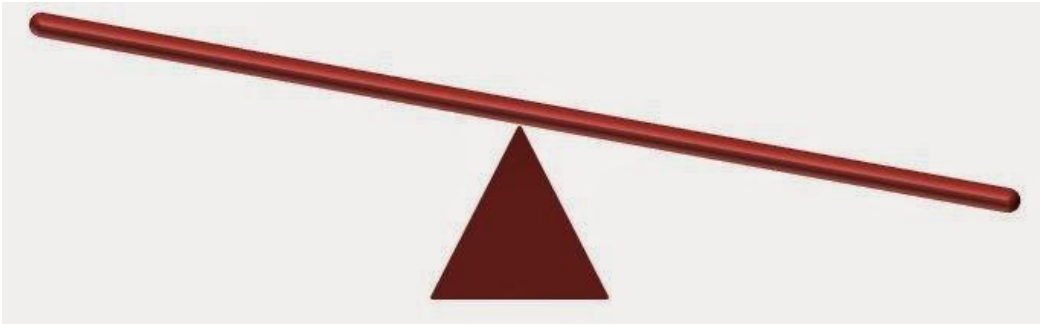
¿Qué afecta y qué la determina?

- La arquitectura de software afecta la
  - Performance
  - Seguridad
    - Safety: Protección contra peligros y accidentes no intencionales.
    - Security: Protección contra amenazas y actos intencionales maliciosos.
  - Disponibilidad
  - Mantenibilidad
  - …

- La Arquitectura de un sistema está fuertemente ligada a los requerimientos no funcionales.

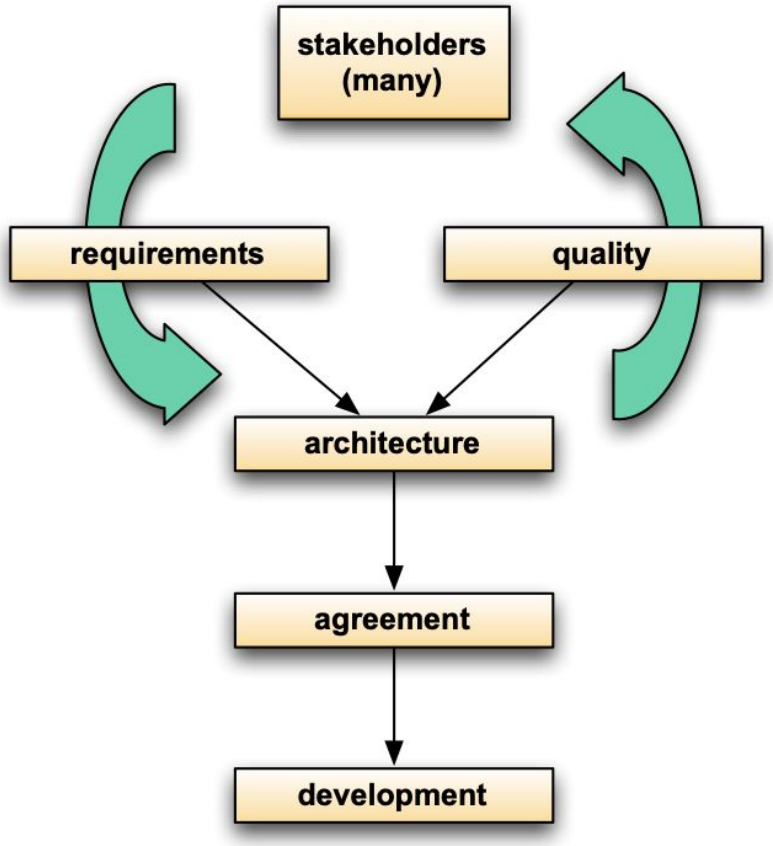
Conflictos entre Soluciones

- El sistema debe presentar **alta performance** y de **fácil mantenimiento** (mantenible).
- ¿Cuál es el conflicto al momento de elegir el estilo arquitectónico?
- ¿Cómo se puede solucionar?
  - Solución de compromiso
  - Diferentes estilos para distintas partes del sistema.



La Arquitectura como solución de compromiso

- La arquitectura ofrece una alternativa de solución que los interesados influencian, deben comprender y validar las características que determina de la Solución.

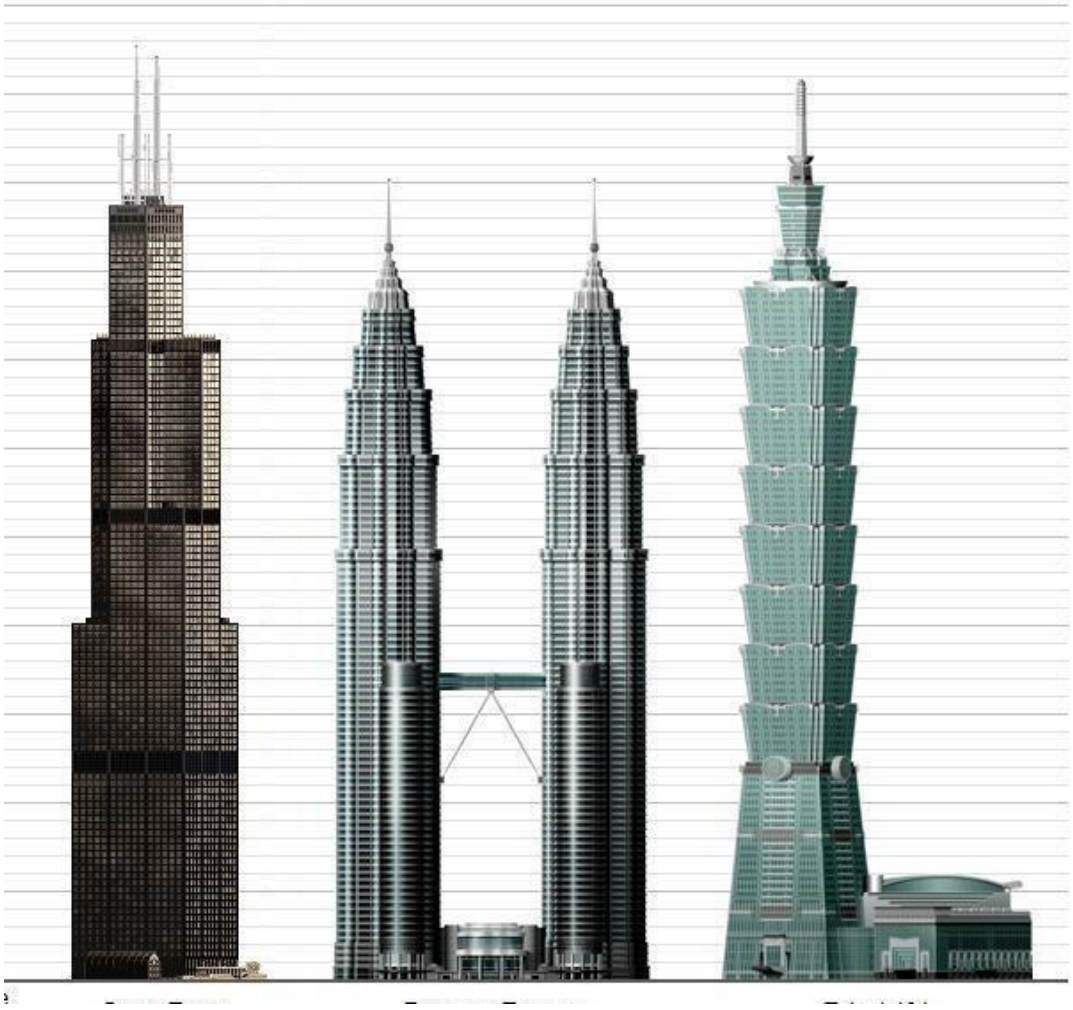


¿Qué tan fácil es de Modificar?

Sears EEUU 527 m.

Petronas Malasia 452 m.

Taipei 101 China 508 m.



¿Qué tan fácil es de Modificar?

Algunos interesados quisieran trasladar ascensores al otro lado …

Los inversores quisieran que el puente estuviera 20 pisos más arriba para una mejor vista.



Arquitectura y Metodologías Agiles

- ¿Hay conflicto entre Arquitectura de Software y las Metodologías Agiles?
  - Agile: Equipos autónomos, evolución permanente
  - Arquitectura: Plan a priori y restricciones
- No son opuestas, pero aplicar ambas requiere tomar medidas.
  - Arquitectos como Interesados.
  - Arquitectos incorporados a los equipos.
  - Modelos de referencia preacordados.

Patrones de Software

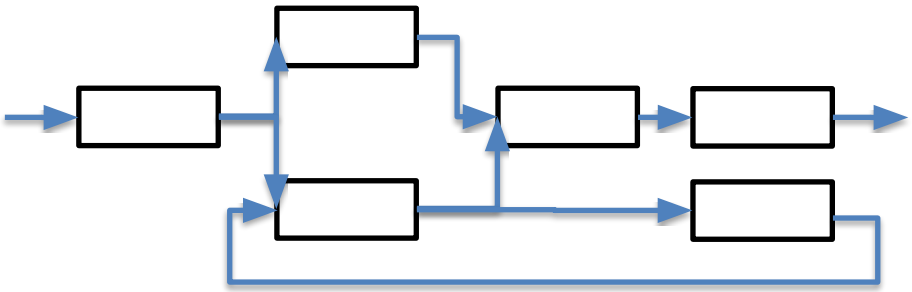
- Propósito
  - Compartir una solución probada, ampliamente aplicable a un problema particular de diseño.
  - El patrón se representa en una forma estándar que permite que sea fácilmente reutilizado.
- Elementos de un patrón
  - Nombre
  - Contexto
  - Problema que considera
  - Solución
  - Consecuencias (positivas y negativas)

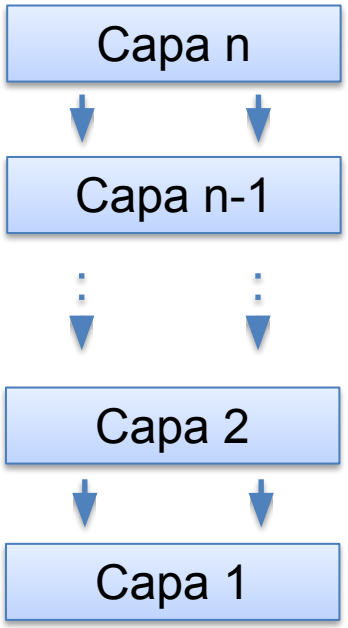
Estilos, Patrones e Idioms

- Los patrones de diseño se agrupan en tres tipos:
  - **Estilos arquitectónicos**: Soluciones de organización a nivel del sistema.
  - **Patrones de diseño**: Soluciones a problemas detallados de diseño de software.
  - **Idioms**: Soluciones útiles para problemas específicos en algún lenguaje de programación.

Estilos Arquitectónicos

- Expresan un esquema de organización estructural para sistemas de software.
- Provee un conjunto de tipos de elementos predefinidos, especifica sus responsabilidades e incluye reglas y guías para organizar las relaciones entre ellos.





Patrones de Diseño

- Un patrón de diseño
  - Provee un esquema para refinar los elementos de un sistema de software o las relaciones entre ellos.
  - Describe una estructura recurrente de elementos de diseño interconectados que soluciona un problema general de diseño dentro de un contexto particular.

Idioms

- Un idiom
  - Es un patrón de bajo nivel, específico para un lenguaje de programación.
  - Describe cómo implementar aspectos particulares de elementos o de las relaciones entre ellos usando las características de un lenguaje particular.

Arquitectura de Software

Estilos Arquitectónicos

Beneficios de aplicar Estilos de Arquitecturas

- Reúso
  - Permite seleccionar una solución entendible y probada en ciertos problemas, definiendo los principios organizativos del sistema.
    - Conocimiento y Experiencia Previa
    - Técnicas y Herramientas
- Comunicación
  - Al basar la arquitectura en estilos que son conocidos, las personas entienden más fácilmente las características importantes de la misma.

Formas de Uso de los Estilos

- Solución para el diseño del sistema
  - Algún estilo sirve
  - Se adopta y se usa como una de las estructuras centrales de la arquitectura.
- Base para una adaptación
  - Algún estilo soluciona parcialmente los problemas.
  - Se puede buscar adaptar el estilo para las restricciones particulares que se enfrentan.

Formas de Uso de los Estilos (2)

- Inspiración para una solución relacionada
  - Ningún estilo sirve.
  - Sin embargo, entender problemas que solucionan ciertos estilos puede llevar a entender mejor el problema actual.
  - Entonces, se puede encontrar una solución que de alguna forma está relacionada con estilos existentes.
- Motivación para un nuevo estilo
  - El problema no es atacado por ningún estilo considerado.
  - Es una buena oportunidad para encontrar una solución general al problema y crear un nuevo estilo.

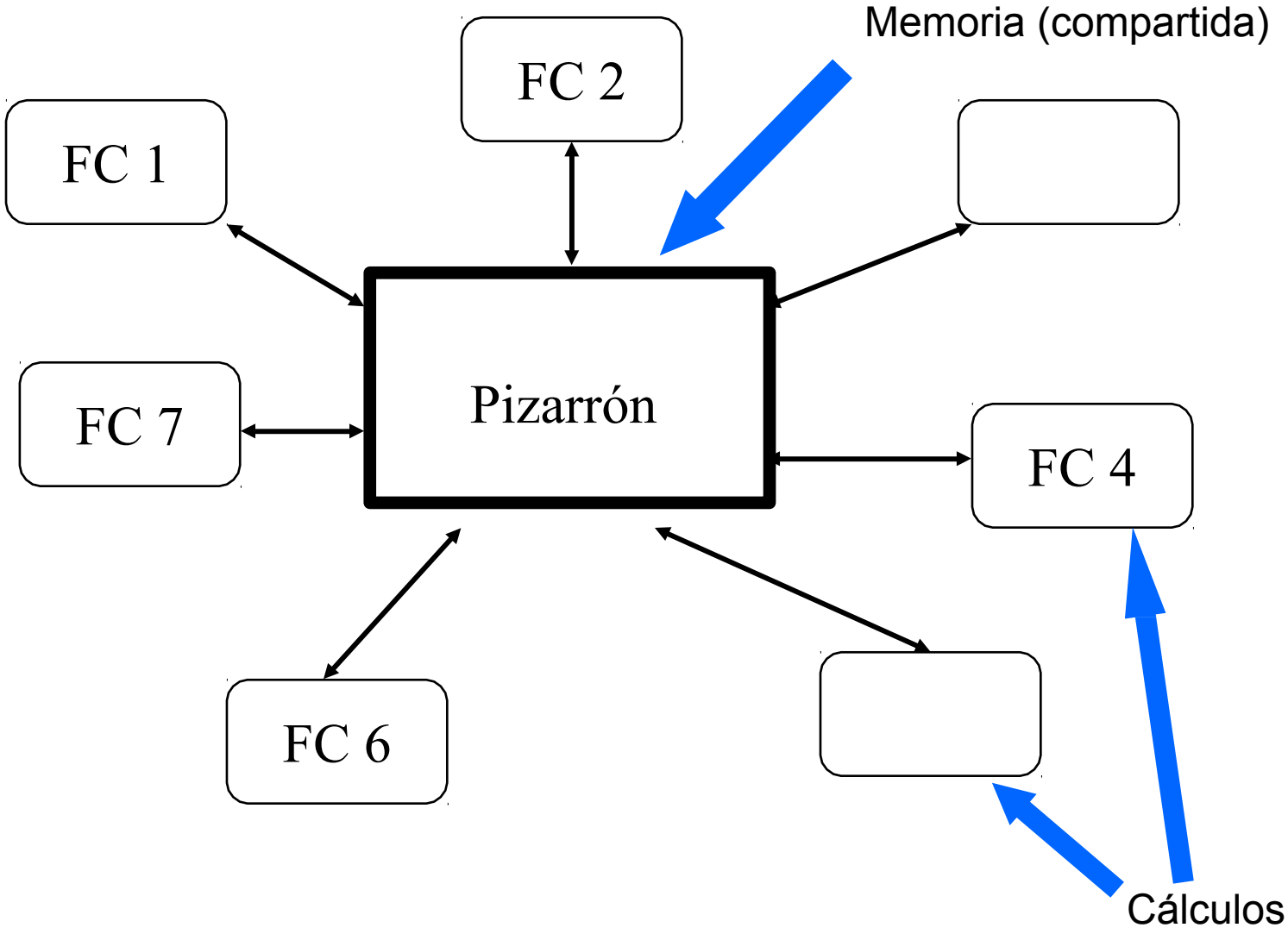
Algunos Estilos

- Cliente – Servidor
- Capas Jerárquicas
- Descomposición Orientada a Objetos
- Tubos y Filtros (pipes & filters)
- Control Centralizado
- Control Basado en Eventos
- Arquitecturas de Sistemas Distribuidos
  - Cliente – Servidor
  - Objetos Distribuidos
  - Peer to Peer
  - Service Oriented Architecture (SOA)
  - Rest & Microservices

Shared-Data

- Generalidades
  - Almacenamiento central de datos y un conjunto de componentes que operan sobre éste.
  - Interacción – intercambio de datos persistentes
  - Múltiples accesos a los datos
  - Al menos un almacenamiento compratido
    - Variaciones
      - Repositorio: el consumidor busca los datos
      - Pizarrón (blackboard): Se avisa al consumidor del arribo de nuevos datos de interés.

Pizarrón (blackboard)



Shared-Data: Ventajas y Desventajas

- Forma eficiente de compartir grandes cantidades de datos.
  - No se transmiten datos de un componente a otro.
- Sin embargo, los subsistemas deben estar de acuerdo en el modelo de datos del repositorio.
- Las componentes que producen datos no necesitan conocer cómo van a ser usados esos datos.
- Actividades como respaldos, seguridad, control de acceso y recuperación están centralizadas.
- Sin embargo, diferentes componentes podrían requerir distintas políticas de recuperación, respaldo, etc.

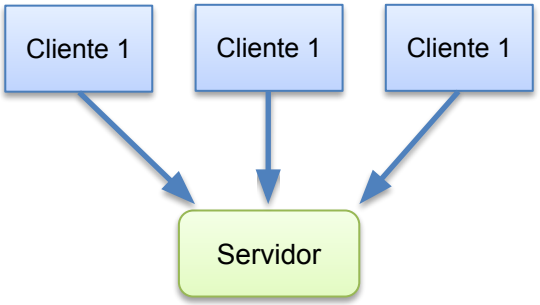
Cliente - Servidor

- El sistema se descompone en dos capas: servidores y clientes.
  - Servidores que ofrecen servicios.
  - Clientes que usan servicios ofrecidos por los servidores.
  - Mecanismo de comunicación entre Servidores y Clientes.

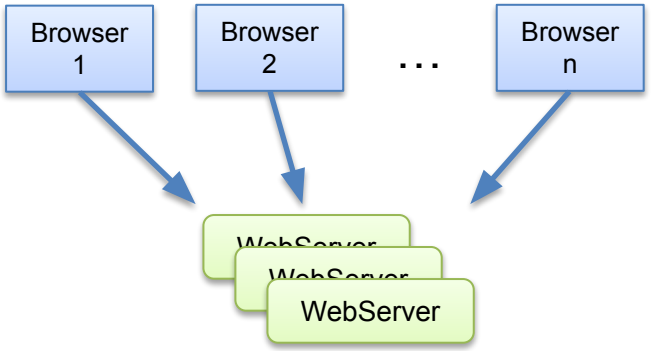
Los clientes interactúan con los servidores, pero no directamente con otros clientes.
Los servidores no inicial interacciones, responden a requerimientos de clientes

Cliente – Servidor (2)

Estilo

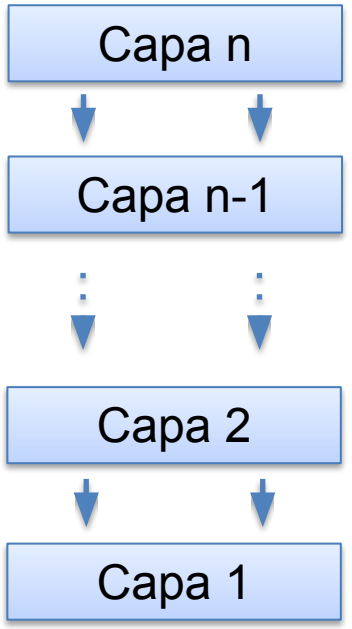


Ejemplo con múltiples servidores



Capas Jerárquicas

- El sistema se organiza en capas.
- Cada una provee un conjunto de servicio a las capas superiores y requiere servicios de las capas inferiores.



Capas Jerárquicas (2)

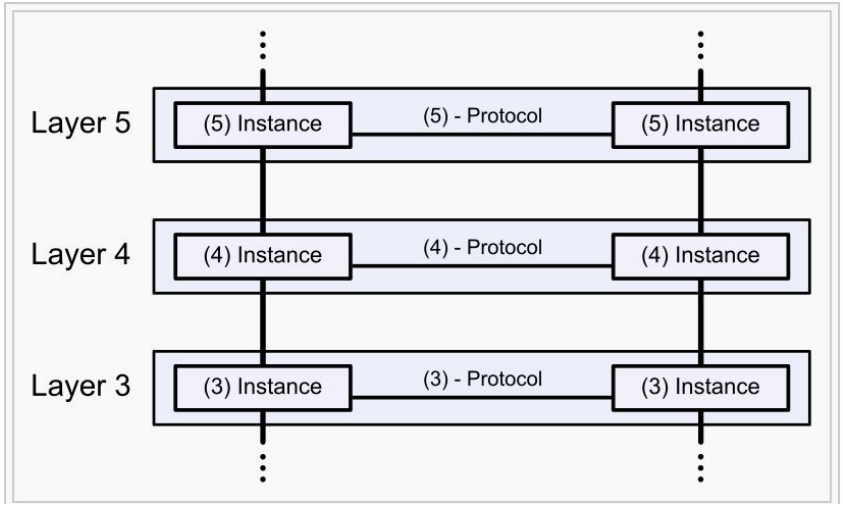
- Modelo estricto
  - Una capa sólo utiliza servicios de la inmediata inferior.
- Modelo relajado
  - Se pueden ‘saltar’ capas.
- Ejemplo:
  - Modelo OSI

OSI model
by layer

7.	**Application layer**	[show]
6.	**Presentation layer**	[show]
5.	**Session layer**	[show]
4.	**Transport layer**	[show]
3.	**Network layer**	[show]
2.	**Data link layer**	[show]
1.	**Physical layer**	[show]

V • T • E

Algo particular de este ejemplo: las capas ‘conversan’ (ejecutan un protocolo) con otra capa en su mismo nivel en otro nodo.



Capas Jerárquicas (3)

- Ventajas
  - Facilita la comprensión
  - Facilita mantenimiento
  - Facilita reutilización
  - Facilita portabilidad
- Desventajas
  - No siempre es fácil estructurar en capas ni identificar los niveles de abstracción a partir de los Requisitos.
  - La interpretación de comandos en múltiples niveles puede afectar el desempeño.

Descomposición Orientada a Objetos

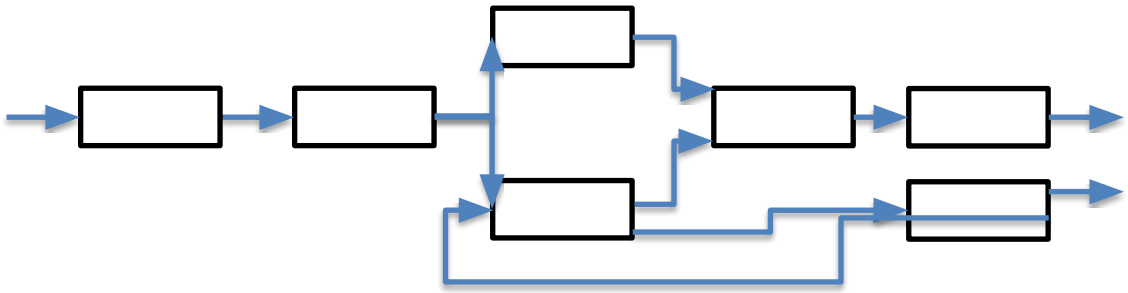
- Se descompone el sistema en un conjunto d objetos que se comunican.
- Representación de datos y operaciones asociadas se encapsulan en cada objeto.
- Elementos típicos en este estilo:
  - Interfaces, herencia, polimorfismo, enlaces dinámicos.

Descomposición Orientada a Objetos (2)

- Ventajas
  - La implementación de los objetos puede ser cambiada sin afectar a otros.
  - Promueve la reutilización de componentes.
  - Muchos objetos representan entidades de la realidad por lo que es fácil entender la estructura del sistema.
- Desventajas
  - Para usar servicios se debe conocer el nombre de la interface de otro objeto.
  - Los cambios en las interfaces afectan a todos los objetos que la usan.

Tubos y Filtros (Pipes & Filters)

- Se descompone el sistema en módulos funcionales
- Interacción: sucesiva transformación de un flujo de datos.
- Los datos llegan a un ‘filtro’, se transforma y son pasados a través de ‘tubos’ al siguiente filtro.
- Un filtro puede pasar datos a múltiples tubos y recibir datos de múltiples tubos.
- Cada filtro es independiente del resto y no conoce la identidad de los otros filtros.
- La transformación del filtro puede comenzar antes de terminar de leer la entrada.
- Respetando el grafo, no importa la secuencia (paralelismo)



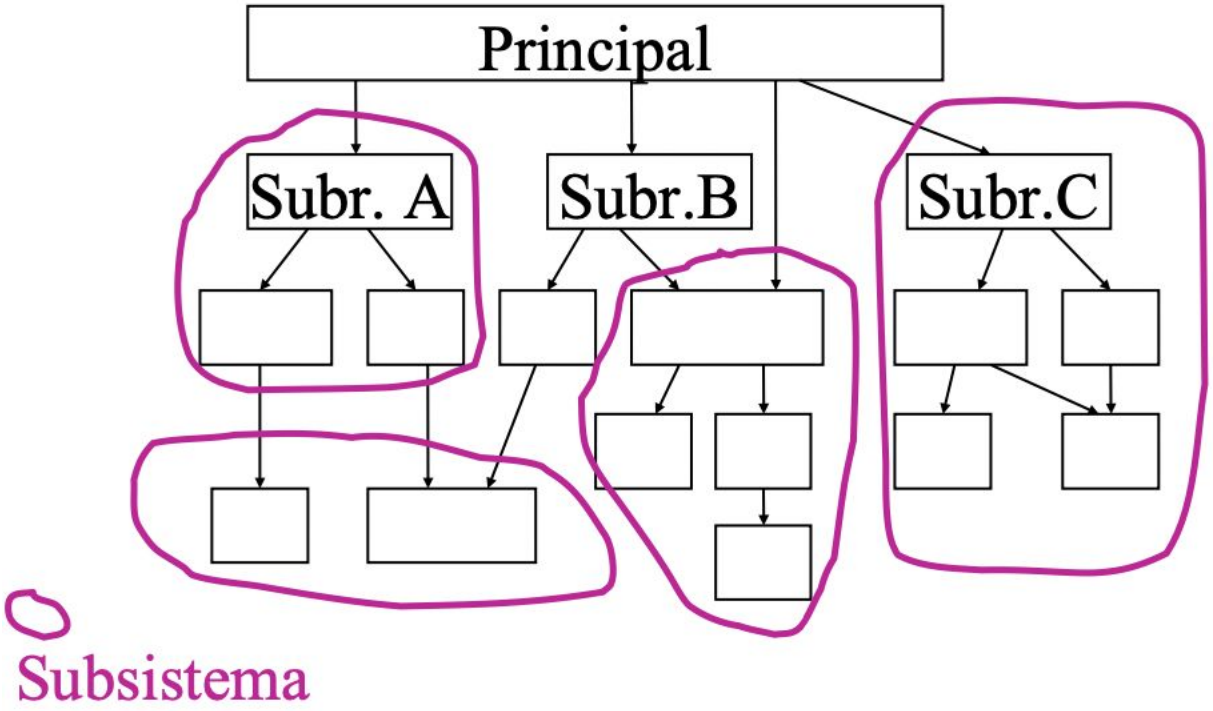
Tubos y Filtros (2)

- Ventajas
  - Se pueden reutilizar los filtros.
  - Es intuitivo pensar en secuencias de procesamientos de datos.
  - Agregar nuevas transformaciones de forma que el sistema evoluciones es sencillo.
- Desventajas
  - Acordar cuál es el formato de los datos.
  - Sistemas interactivos son difíciles de construir con este estilo.
  - Ineficiente si hay procesamiento (por la descomposición) o si los filtros repiten chequeos.

Modelos de Control

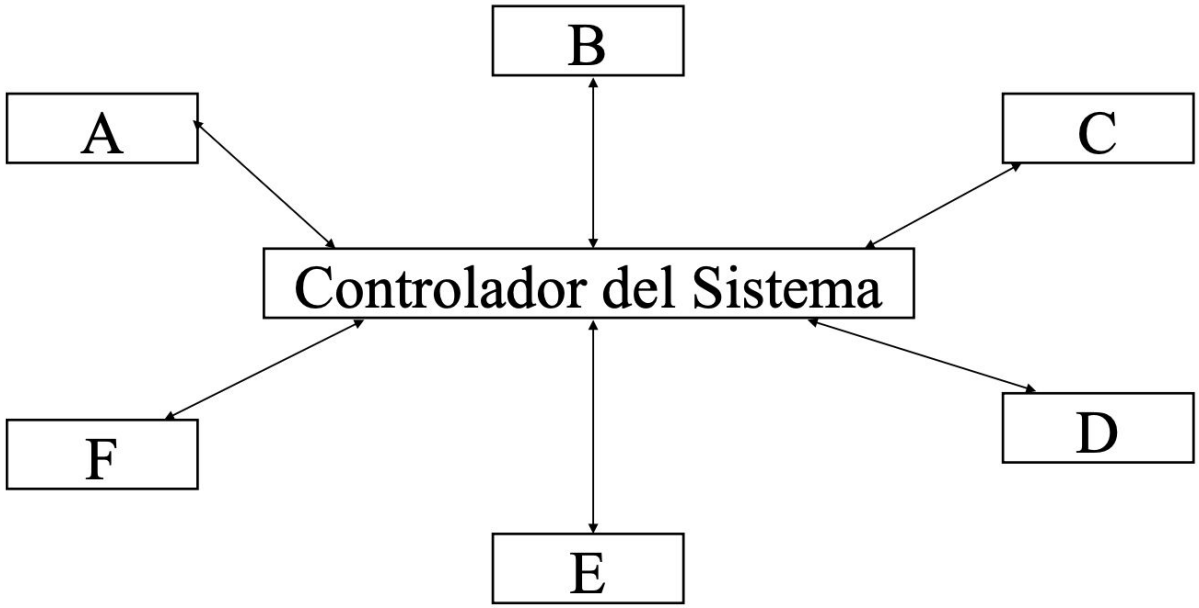
- Modelos de control a nivel de la arquitectura se preocupa del flujo de control entre subsistemas.
- Control centralizado
  - Un subsistema controla al resto
- Control basado en eventos
  - Cada subsistema puede responder a eventos externos
    - Eventos generados por otros subsistemas
    - Eventos generados por el ambiente del sistema

Control Centralizado
Llamada/Retorno (request/response)



Control Centralizado
Modelo Gerente (manager)

- Un componente controla los procesos de otros componentes.



Control basado en Eventos

- En los modelos centralizados las decisiones de control suelen estar determinadas por valores de alguna variable de estado del sistema.
- En cambio, el control basado en eventos es guiado por eventos generados externamente.
- Ejemplos
  - GUIs: eventos representan acciones externas.
  - Shutdown: Se emiten eventos a todos los subsistemas (broadcast) para que detengan su operación.

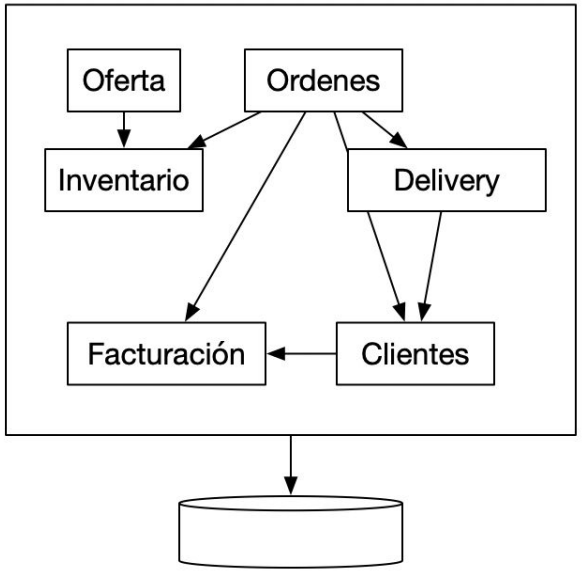
Publicación/Subscripción (publish/subscribe)

- Sistema de componentes independientes que se comunican mediante mensajes/eventos:
  - Los que publican no saben cuántos consumidores hay registrados para el consumo.
  - Los subscriptores indican en qué mensajes están interesados.
- Interacción vía eventos/mensajes.
  - Tópicos: una alternativa para establecer ‘temas’ para publicar o subscribirse.
- Sistema de Mensajería / Middleware
  - Es trabajo de esta plataforma asegurar la comunicación de los mensajes a todos subscriptores bajo distintas condiciones (transaccionales, retención, reintentos, etc.)

Arquitectura Monolítica (monolithic)

- Aplicación desplegada como unidad indivisible, que no es responsable de una única función.
- Ventajas
  - Desarrollo ágil en etapas tempranas.
  - Baja complejidad de comunicación.
- Desventajas
  - Cambio de un módulo requiere reconstruir y desplegar toda la solución.
  - Limitaciones para escalar.
  - Posible baja flexibilidad.

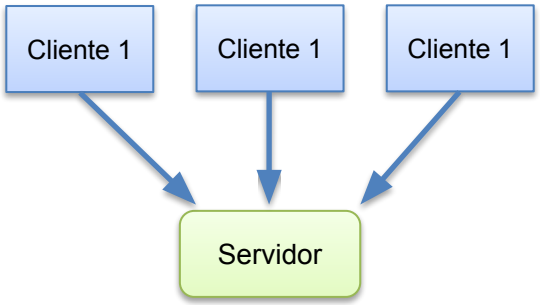




Arquitectura de Sistemas Distribuidos

Cliente - Servidor

- Más de una forma de distribuir la lógica entra las capas.
- El enlace ahora es sobre una red, con sus consecuencias.



Cliente-Servidor
2 Niveles

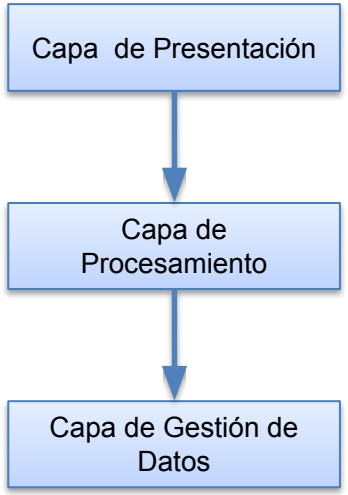
- Es la arquitectura más simple cliente-servidor.
- Varios clientes y un servidor
  - Es posible usar múltiples servidores idénticos para escalabilidad / tolerancia a fallas.
- Es posible balancear funcionalidad entre las capas
  - Cliente fino
    - El procesamiento de la información y la gestión de los datos ocurre en el servidor.
    - El cliente sólo es responsable de ejecutar el software de presentación.
  - Cliente grueso
    - El servidor es responsable sólo de la gestión de los datos.
    - El cliente ejecuta la lógica de la aplicación y la presentación.

Cliente – Servidor en 2 niveles
Algunas limitaciones

- Hay que distribuir presentación, lógica y datos en dos capas o ‘nodos’.
  - Cliente fino:
    - Presentación separada de resto
    - Escalabilidad ?, rendimiento?
  - Cliente grueso:
    - Presentación y lógica separada de los datos
    - Administración del sistema?, rendimiento.
- Alternativa
  - Utilizar más niveles.
  - 3 o 4 niveles son típicos.

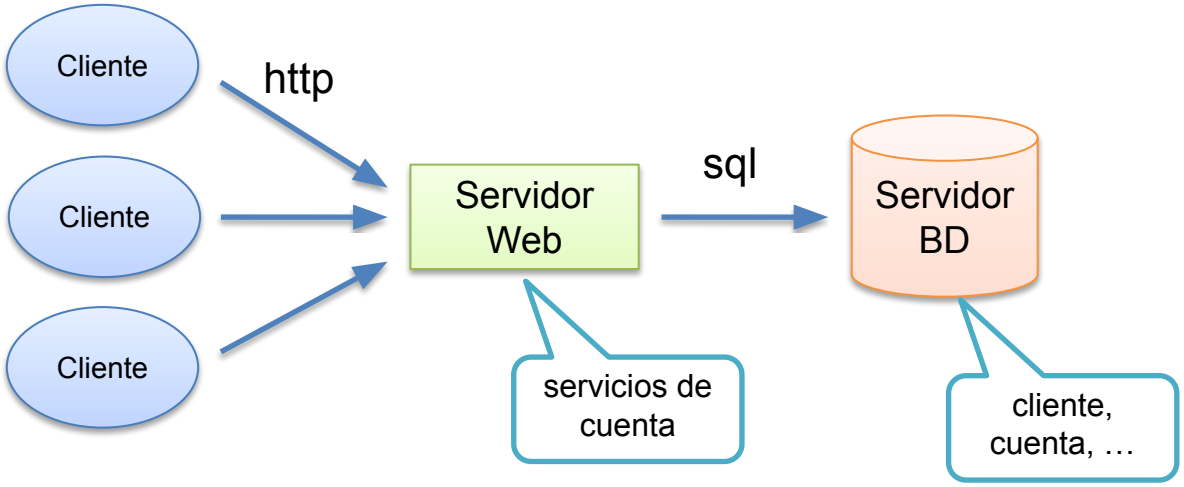
Cliente-Servidor en 3 Capas

- Mayor flexibilidad para distribuir la funcionalidad del sistema.
- Presentación
  - Presentación de información e interacción con el usuario.
- Procesamiento
  - Implementación de la lógica de la Aplicación
- Gestión de Datos
  - Operaciones de BD y archivos



Cliente-Servidor en 3 Capas

- Las capas pueden asignarse a distintos nodos de cómputo (ej. máquinas).
  - Ejemplo: Banco x Internet (home banking)



Comparación

Cliente Fino vs 3 capas

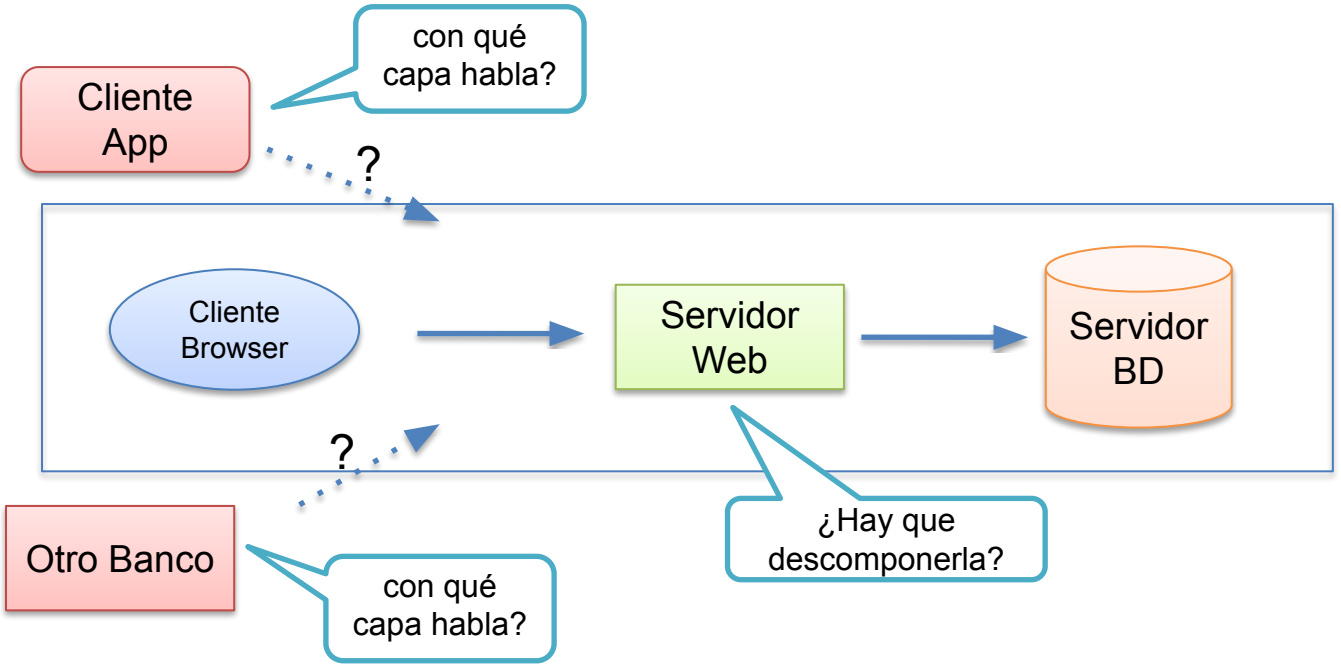
- Cliente Fino tiene mayor tráfico de red
- 3 capas
  - escala con mayor facilidad

Cliente Grueso vs 3 capas

- Cliente Grueso con mayor complejidad de gestión
- 3 capas
  - mantiene lógica con gestión centralizada
  - más fácil de actualizar

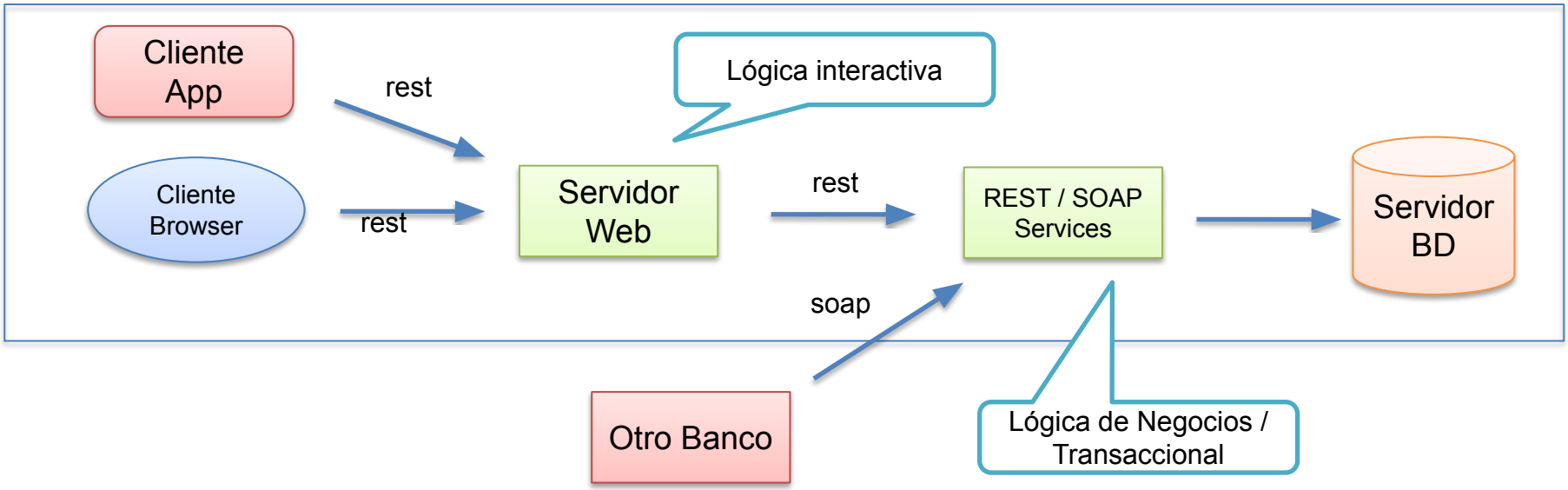
3 Capas - Limitaciones

- ¿Cómo integrar distintos tipos de Clientes?
  - Ej. Web Browser y Apps.
- ¿Cómo ofrecer Servicios de Cuenta a Otros bancos?
  - Ej. Web Services



4 Capas - Ejemplo

- ¿Cómo integrar distintos tipos de Clientes?
  - Ej. Web Browser y Apps.
- ¿Cómo ofrecer Servicios de Cuenta a Otros bancos?
  - Ej. Web Services



Objetos Distribuidos

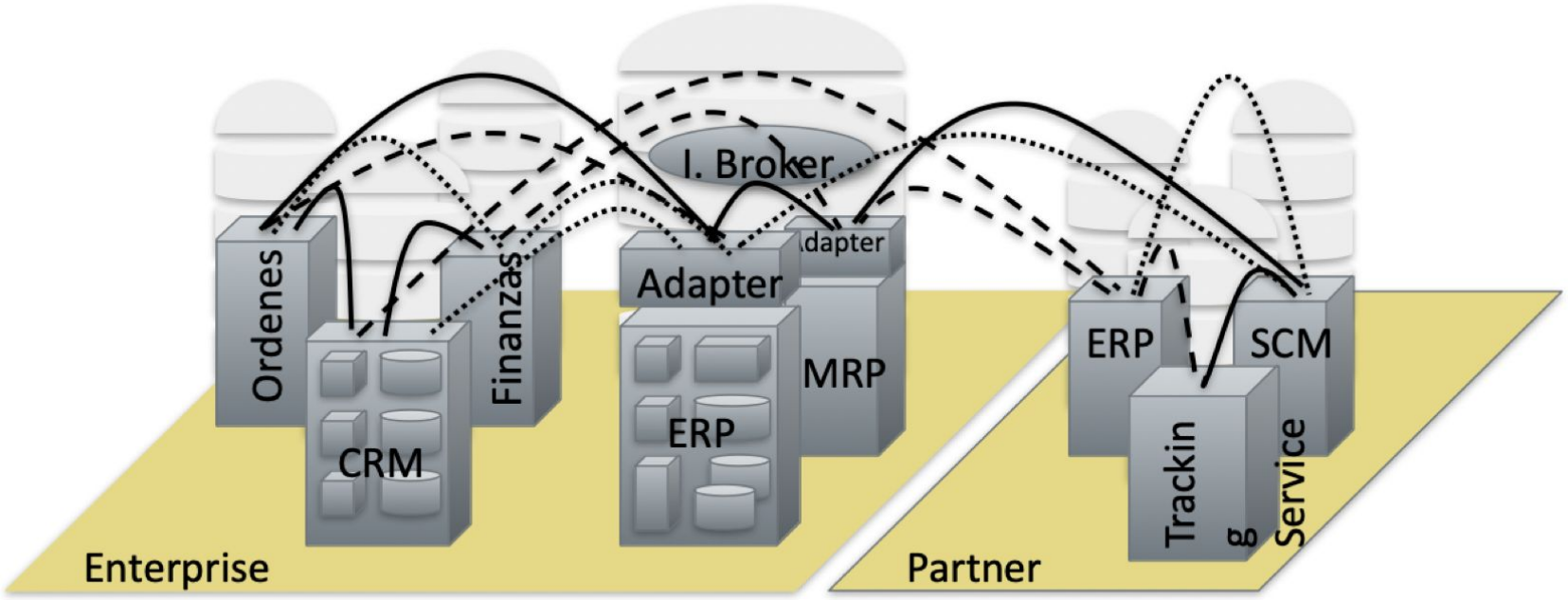
- En la arquitectura cliente-servidor los clientes y los servidores son distintos
  - Los servidores reciben servicios de los clientes y no de otros clientes.
  - Los servidores pueden actuar como clientes de otros servidores, pero no requieren servicios de clientes
  - Los clientes deben conocer los servicios ofrecidos por los servidores específicos y deben conocer cómo contactar a esos servidores.
- Enfoque más general
  - Remover la distinción entre clientes y servidores
  - Diseñar la arquitectura como una de objetos distribuidos.

Objetos Distribuidos (2)

- Objetos que proveen / consumen servicios de otros objetos
  - No hay distinción entre clientes y servidores
- Pueden ser distribuidos entre distintos nodos de una red y se comunican a través de un *middleware*.
  - En este caso, el middleware se llama: Object Request Broker (ORB).
  - Corba es un caso.
- Es más complejo de diseñar que cliente-servidor.

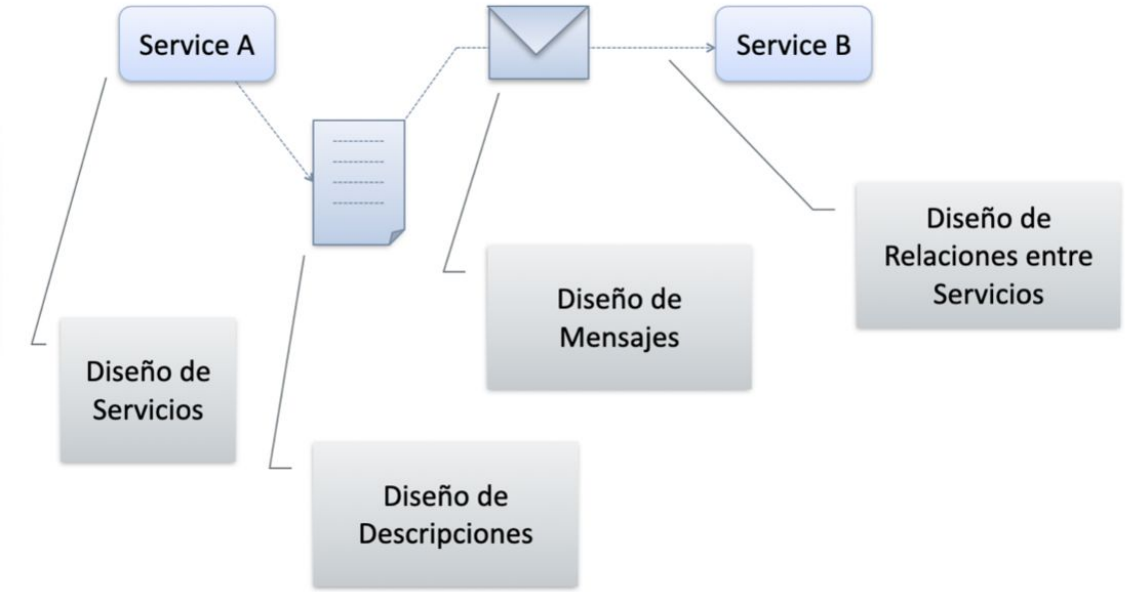
La ‘Arquitectura Accidental’ Anti-patrón

Motivación para SOA



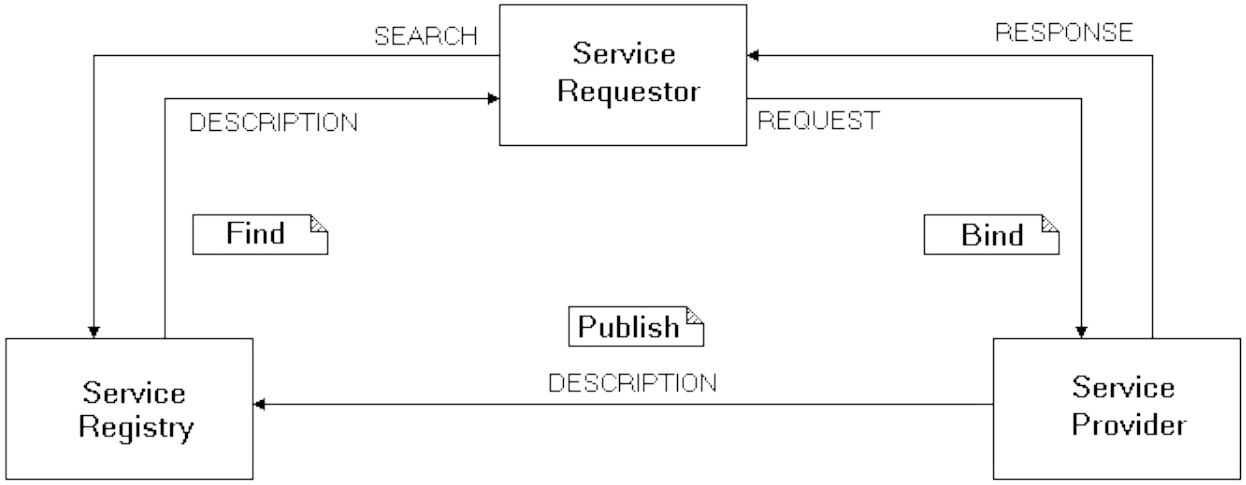
Service Oriented Architecture SOA

- El sistema es el resultado de la composición de Servicios.
  - Un servicio es una unidad de lógica de una solución a la que se le ha aplicado la orientación a servicios en una medida significativa (Thomas Erl).



Service Oriented Architecture SOA (2)

- El servicio es independiente de las aplicaciones que lo usan.
- Aplicaciones distintas pueden compartir servicios.
- Hay distintas formas de llevar Servicios a la práctica, conceptualmente operan de acuerdo con el siguiente modelo:



Servicio Web y Estándares

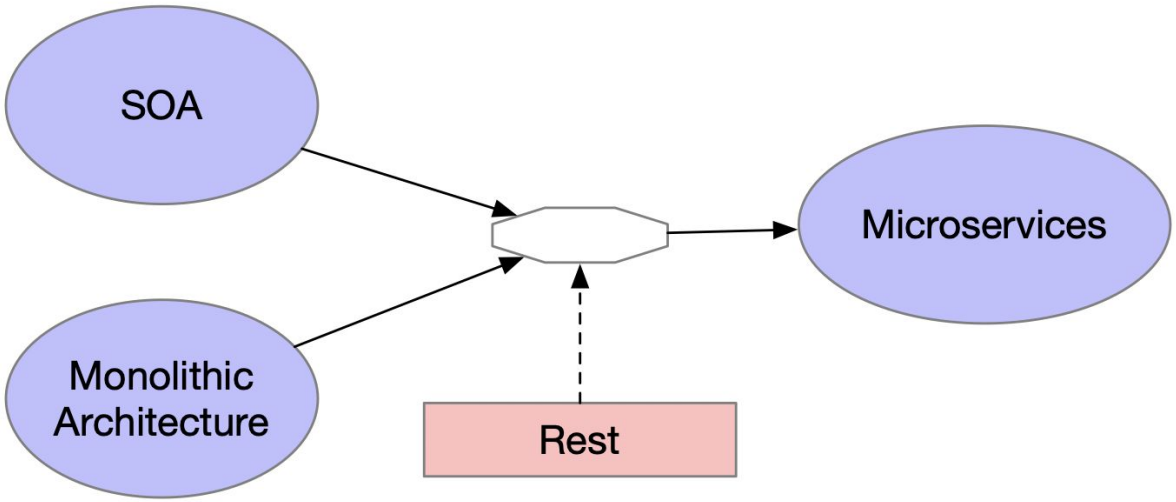
- Un servicio web (web service) es una tecnología que utiliza un conjunto de protocolos y estándares para intercambiar datos enter aplicaciones.
  - WSDL: Interfaz del servicio
  - SOAP: Mensajes intercambiados
  - XML: Serialización de los mensajes
  - HTTP: Se intercambian sobre este protocolo

Estilo REST de Arquitectura

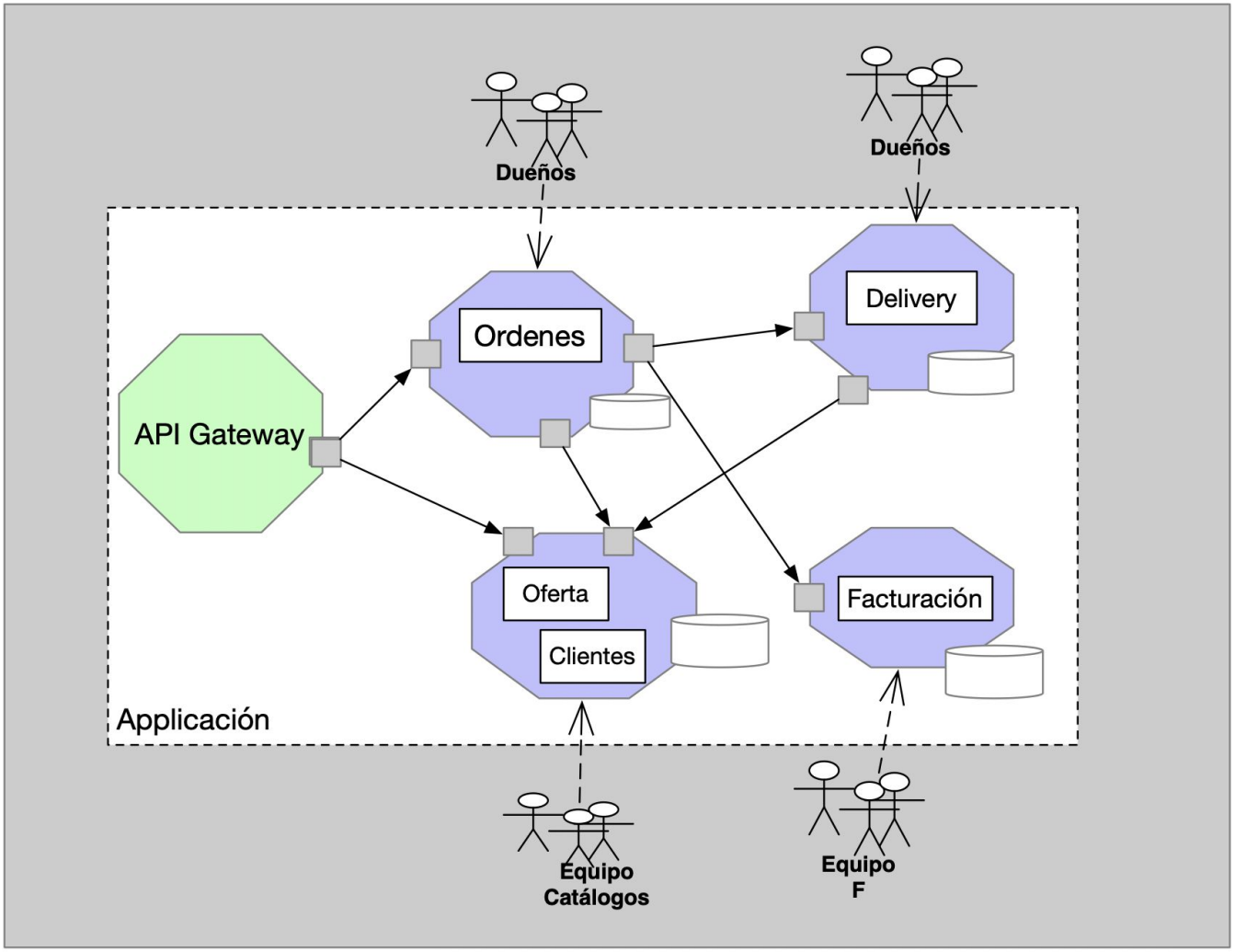
- Se basa en establecer un conjunto de reglas para describir y acceder a recursos en una red.
  - Architectural Styles and the Design of Network-based Software Architectures, Roy Fielding (2000, PhD tesis).
  - Influencia el desarrollo de la WWW.
- Restricciones
  - Cliente-Servidor
  - Stateless (sin estado)
  - Cacheability
  - Layered System (capas)
  - Code on Demand
  - Uniform Interface: Recursos y métodos: GET, POST, PUT.
- RESTful Web Services – RESTful APIs
  - Web Services que promueven conceptos de REST.
  - APIs basadas en HTTP.
  - No todo RPC sobre HTTP es REST (Richardson Maturity Model)

Microservicios

- Un sistema complejo que se modela como un conjunto de unidades desplegadas individualmente favoreciendo pipelines de CI/CD.
- Se lo puede ver como evolución de conceptos de Arquitectura monolítica y SOA combinados y aplicando REST.



Microservicos (2)



Descomposición del Sistema

Descomposición del Sistema

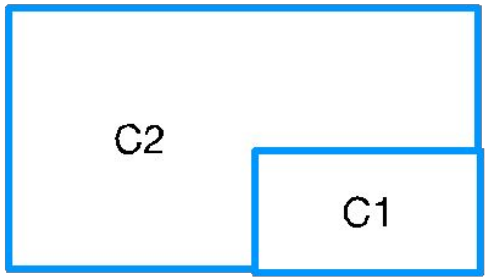
- A nivel de la arquitectura, el foco se pone en los componentes a gran escala del sistema.
- La **descomposición** implica analizar estos componentes a gran escala y representarlos como un conjunto de componentes de grano más fino.

Servicio, Componente, Módulo

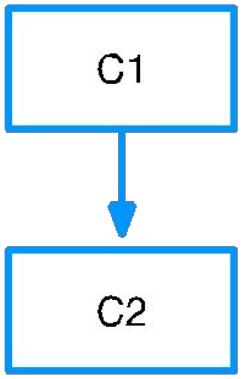
- Servicio
  - Unidad coherente de funcionalidad
    - Ej. Servicio para enviar mails
- Componente
  - Unidad de software con nombre que ofrece uno o más servicios a otros componentes o al usuario.
  - Servicios consumidos desde otros componentes lo hacer a través de un API.
- Módulo
  - Conjunto de componentes con nombre que tienen algo en común.

Relaciones entre Componentes

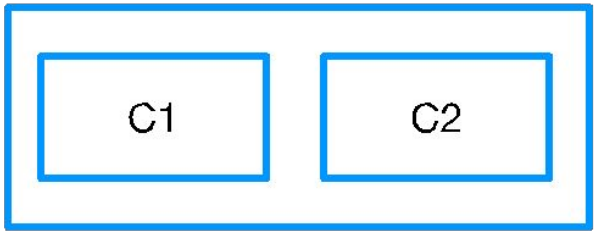
C1 es-parte-de C2



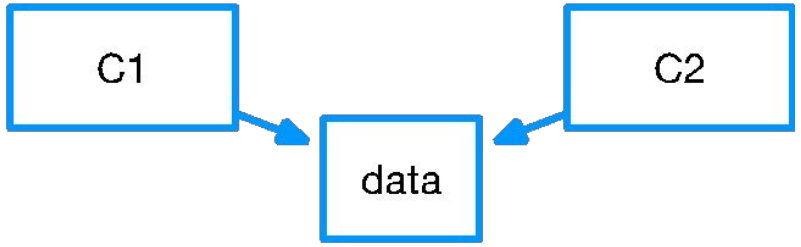
C1 usa C2



C1 es-coubicado-con C2

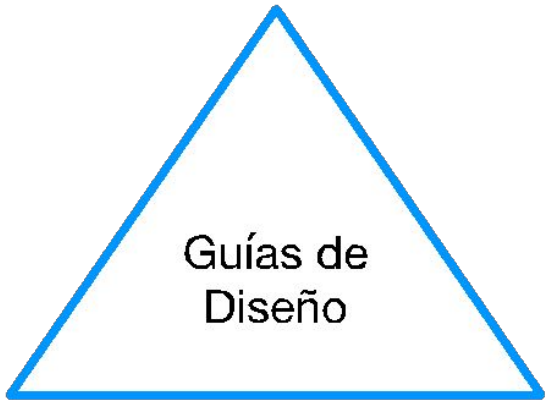


C1 comparte-datos-con C2



Pautas de diseño de la arquitectura

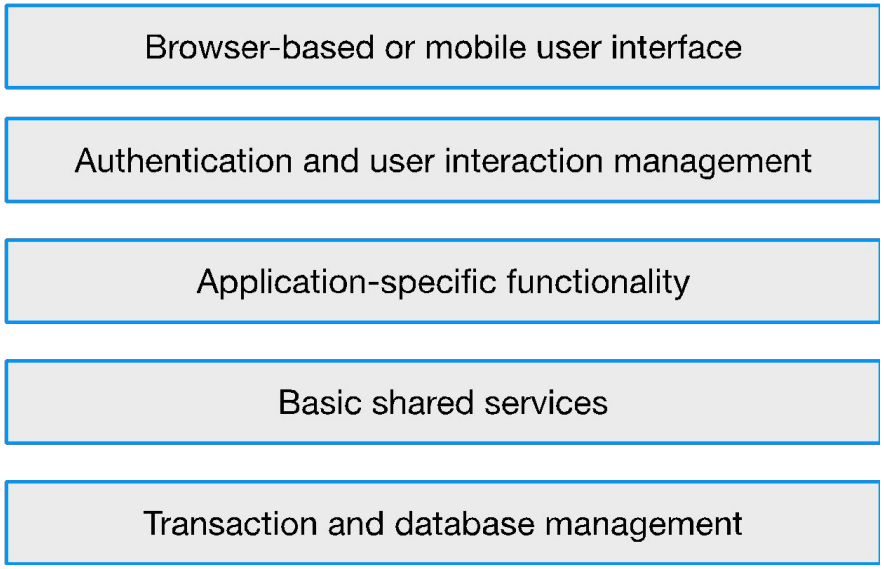
Separación de Temas
Organizar la Arquitectura en componentes que se enfocan en un tema único



Interfaces Estables
Diseñar interfaces de componentes coherentes y que cambien lentamente

Implementar una vez
Evitar duplicar funcionalidad en distintos puntos de la Arquitectura

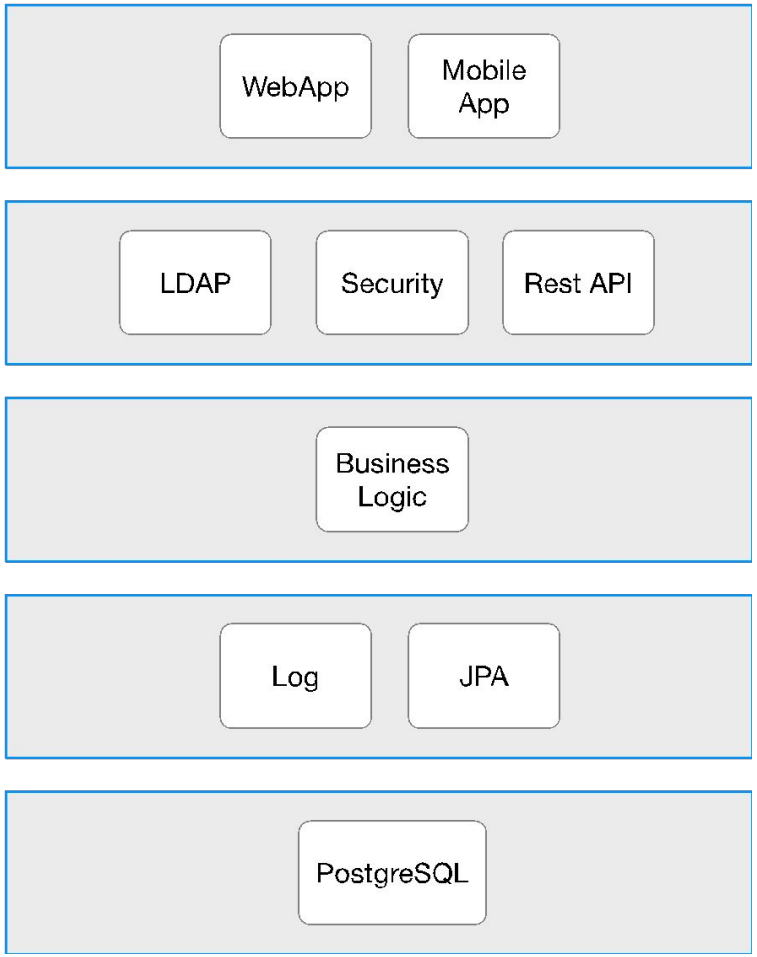
Un possible punto de partida Estructura en Capas



- Este puede ser un enfoque inicial
  - Capas coherentes, componentes con temas comunes
  - El número de capas se debe ajustar al caso
- Aplicar los criterios de diseño relevantes para el sistema particular
  - Refinamiento

Caso: Sistema de Gestión de Pedidos

- Primera arquitectura usando estructura en capas
- ¿Cómo incorporar?
  - Catálogos corporativos
  - Datos no estructurados
  - Búsqueda de pedidos en forma aproximada
  - Interoperación con sistemas corporativos
  - Eventos corporativos
  - Auditoría y Monitoreo
- ¿Características de calidad?
- ¿Cómo afectan decisiones tecnológicas?



Decisiones Tecnológicas

- Bases de Datos
  - Relacionales
  - NoSQL
- Plataforma de Interacción
  - Web
  - Mobile
- Server
  - Servidor dedicado
  - Cloud
- APIs
  - Desarrollados

Decisiones Tecnológicas

- Bases de Datos
  - Relacionales
  - NoSQL ?
- Plataforma de Interacción
  - Web
  - Mobile ?
- Server
  - Servidor dedicado
  - Cloud?
- APIs
  - Desarrolladas
  - API Gateway ?
- Monitoreo
  - ?
- Auditoría
  - ?
- Herramientas de desarrollo

Decisiones de Tecnologías durante el Diseño

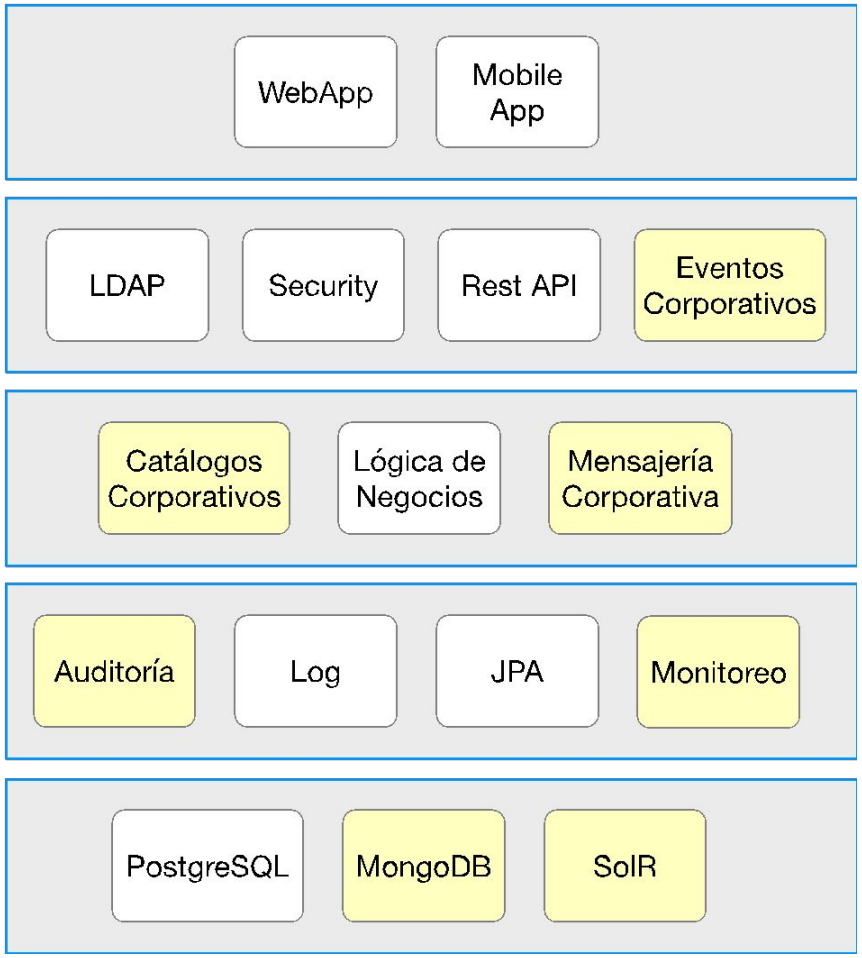
- Influyen y restringen la arquitectura.
- No es fácil cambiar decisiones tecnológicas.
- Bases de Datos
  - Relacionales: Transacciones ACID
  - NoSQL: modelos alternativos de persistencia
- Aplicaciones móviles
  - ¿Complemento o plataforma primaria?
  - Conectividad, capacidad de proceso, manejo de energía, pantalla/teclado.
- Server
  - ¿Dedicado o cloud?

Open Source Software

- El uso de OSS está ampliamente difundido en múltiples sectores del mercado.
- Consideraciones a valorar
  - ¿Reusar OSS o implementar?
  - ¿Disponemos de las capacidades para aplicarlo?
  - ¿Está sobre dimensionado?
  - Licenciamiento apropiado
  - Respaldo de la comunidad
  - Implicancia en decisiones de diseño

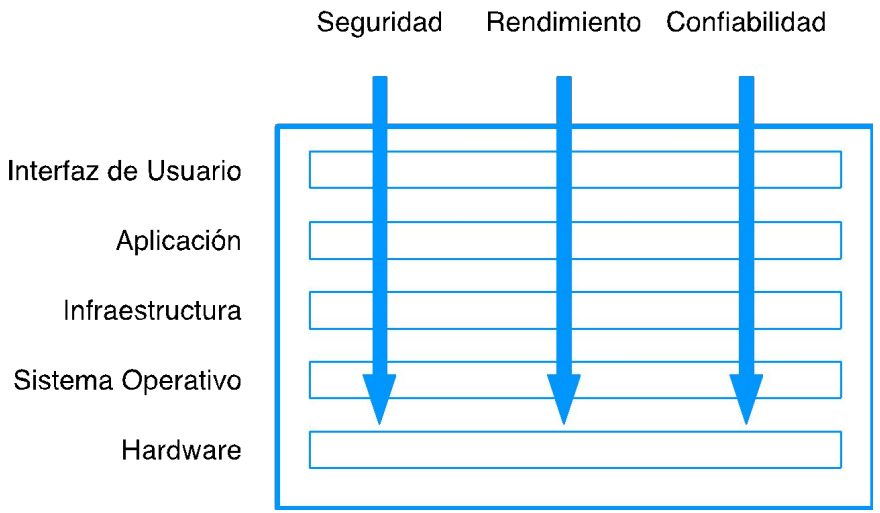
Caso: Extensión de la primera versión de la arquitectura

- La estructura en capas puede ir cediendo.
  - Capas no estrictas
  - API Rest debe acceder a datos, log y monitoreo
  - ¿Distintos temas en la capa de negocios ?
- En qué capa va:
  - ¿Validación?
  - ¿Seguridad?



Consideración de temas transversalmente

- Resolver ‘temas’ en un único punto aumentar riesgos
- Alternativa:
  - Considerarlos a lo largo del sistema (ej. capas)
  - Evitar un único punto de falla.



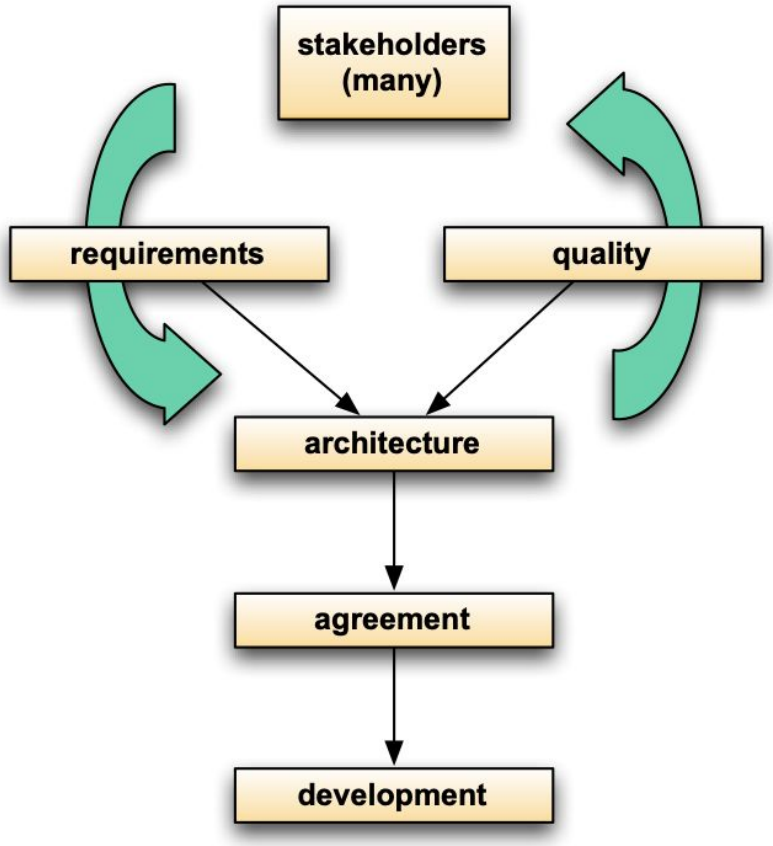
Evaluación y Documentación de Arquitecturas

Evaluación de Arquitecturas

- Cambiar la arquitectura de un producto ya construido requiere mucho esfuerzo.
- Entonces, es importante evaluar la arquitectura antes de implementarla completamente.
  - Verificar los requisitos de calidad establecidos
  - Evaluaciones a posteriori resultan útiles como forma de aprendizaje y estudio de posibilidades de mejora (pe. nuevas versiones).

La Arquitectura como solución de compromiso

- La arquitectura ofrece una alternativa de solución que los interesados influencian, deben comprender y validar las características que determina de la Solución.



Documentación de la Arquitectura

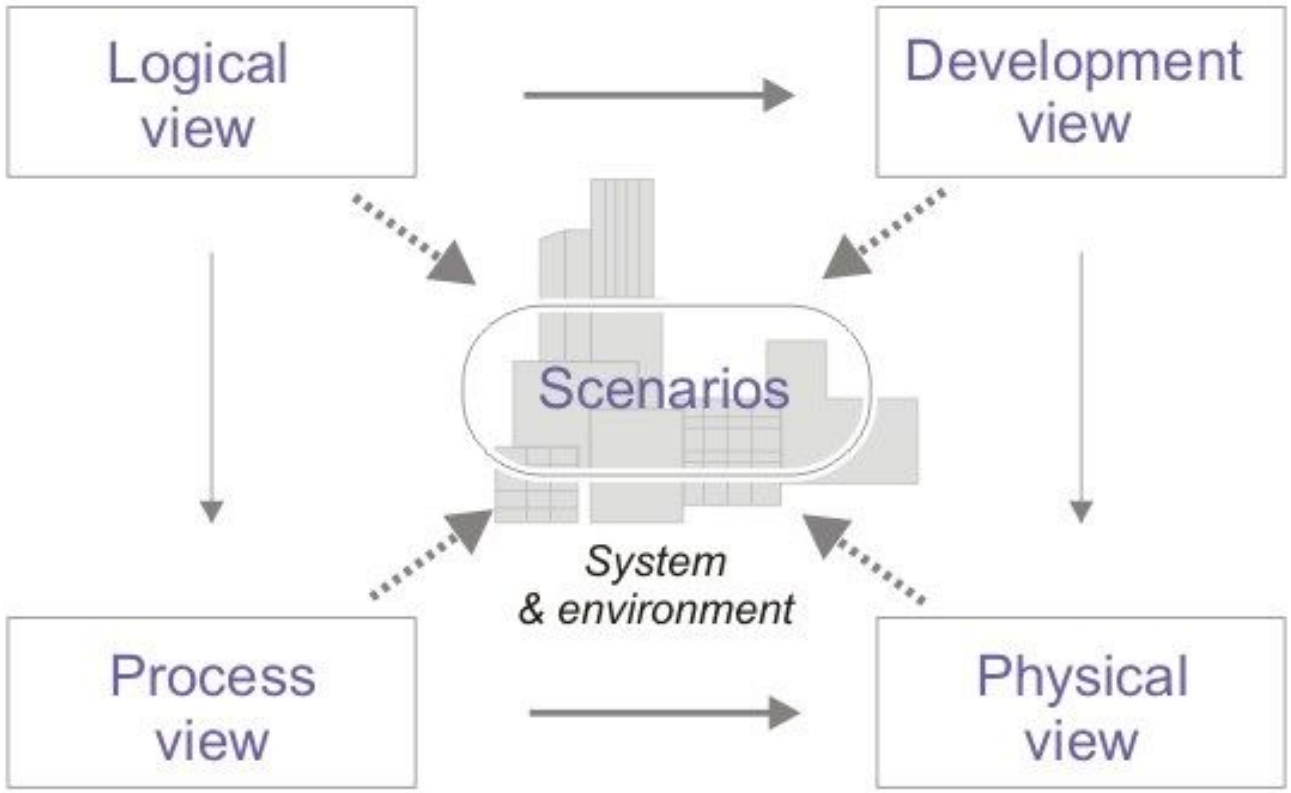
- Objetivo y Destinatarios
  - Interesados que deben entender su impacto y analizar características a incorporar en el sistema.
  - Equipo que construye el sistema.
  - Equipos que mantienen y evolucionan el sistema.
- Contenido
  - Contexto, Decisiones, balance entre alternativas
  - Descripción de Vistas que capturen aspectos relevantes del Sistema desde distintas perspectivas.
- La documentación debería amortizarse sola haciendo que las actividades de desarrollo sean menos costosas.

Documentación de la Arquitectura (2)

- Identificar Audiencia / Interesados
  - Escribir para destinatarios concretos.
- Describir atributos de calidad y justificación de decisiones
  - Patrones / Estilos aplicados
  - Tácticas activadas
  - Alternativas consideradas y descartadas
- Vistas de la Arquitectura
  - Aspectos de la Solución
- Relación entre las Vistas
- Variablidad posible
  - Durante la implementación o evolución.

Ejemplo de uso de Vistas
Modelo de Vistas 4+1

- Propuesta para estandarizar descripción de arquitecturas mediante selección de Vistas.



Modelo 4+1

- Vista lógica
  - abstracciones claves en el sistema como objetos o clases.
  - Ej. diagarmas UML: class, state
- Vista del proceso
  - interacción de procesos en tiempo de ejecución.
  - Ej. diagramas UML: sequence, communication, activity.
- Vista de desarrollo / implementación
  - como el software se descompone para el desarrollo.
  - Ej. diagramas UML: Component, Package
- Vista física
  - hardware del sistema y componentes de software desplegados.
  - Ej. diagramas UML: Deploy
- Escenarios (+1)
  - Relacionados con los casos de uso o escenarios
  - Ej. diagramas UML: Use Case